

Algoritmos y Estructuras de Datos.

- Clase 7 - Computabilidad y Complejidad Computacional -

Complejidad Computacional

- Análisis Experimental
 - Cómo ver cuanto tiempo un algoritmo consume?
 - Más allá del examen experimental.
- Algunas funciones matemáticas importantes
 - Funciones más usadas
 - Comparación del crecimiento de una función
- Análisis Asintótico
 - Notación "Big-Oh".
 - Análisis Comparativo
- Ejemplos

Complejidad Computacional

- Análisis Experimental
- Algunas funciones matemáticas importantes
- Análisis Asintótico
- Ejemplos

Análisis Experimental

Ya escrito un algoritmo, podemos estudiar su tiempo de ejecución, realizando varios tests con diferentes datos de entrada y registrar el tiempo que el algoritmo toma en cada ejecución.

Podemos utilizar la función `time()` provista por Python:

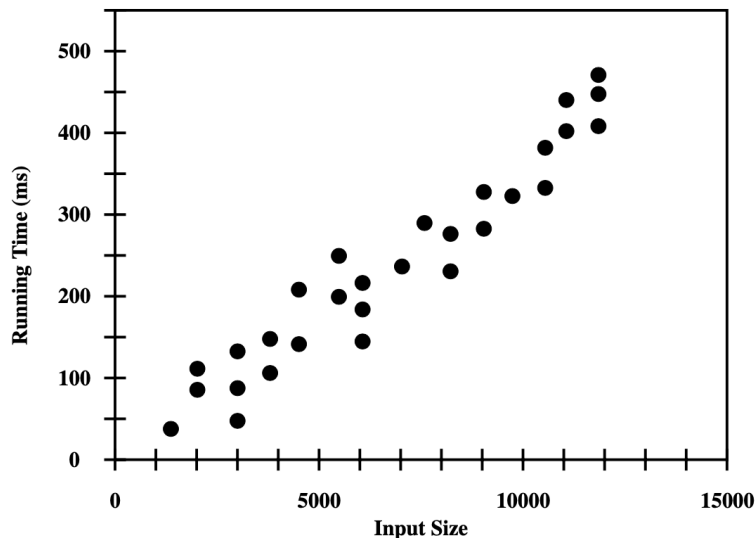
```
In [1]: 1 from time import time
        2
        3 # tiempo de comienzo
        4 start_time = time( )
        5
        6 ## EJECUCIÓN DEL ALGORITMO
        7
        8 #tiempo de finalización
        9 end_time = time( )
       10 # tiempo transcurrido
       11 elapsed = end_time - start_time
```

Este resultado de la función `time()` está expresado en segundos desde **epoch** como un número de coma flotante. La fecha específica de la época y el manejo de los **leap seconds** depende de la plataforma. En Windows y la mayoría de los sistemas Unix, la época es el 1 de enero de 1970, 00:00:00 (UTC) y los segundos bisiestos no se cuentan para el tiempo en segundos desde la época. Esto se conoce comúnmente como Tiempo Unix.

Medir el tiempo de esta forma es un buen punto de entrada, pero no es una solución escalable a nivel general (solo podemos medir casos prácticos y no generales). Cabe destacar, que nuestro algoritmo, en este caso, está compitiendo y haciendo uso compartido del CPU lo cual afectará la "aproximación" de nuestro resultado.

Una mejor métrica, podría ser, calcular el número de ciclos de CPU (instrucciones atómicas) realizadas.

Ejemplo:



Desafíos del Análisis Experimental

Los resultados experimentales son importantes cuando estamos "afinando" un algoritmo considerado para producción. Algunas limitaciones de método:

- Estos tiempos son difíciles de comparar, ya que están restringidos a una "instalación" (hardware/software) en particular.
- Los experimentos están limitados a los "sets de datos de entrada", por ello, no incluyen "todos los posibles casos".
- El algoritmo debe estar "finalizado".

Esta última es la limitación más importante, ya que en las etapas de desarrollo y diseño del algoritmo han finalizado. Es decir, si el algoritmo no es eficiente, debemos comenzar nuevamente.

Más allá del Análisis Experimental

Nuestro objetivo es desarrollar un método para analizar la eficiencia de un algoritmo, tal que:

- Analizar un algoritmo independientemente del hardware o entorno de desarrollo.
- Realizar un estudio de "alto-nivel" sobre la especificación del algoritmo, y no sobre su implementación.
- Toma en cuenta todos los posibles sets de datos.

Cantidad de "operaciones primitivas"

Para analizar el tiempo de ejecución de un algoritmo sin realizar experimentos, debemos analizar la "descripción del algoritmo", ya sea en su implementación en Python, o pseudo-código. Definimos el conjunto de operaciones primitivas como:

- Realizar una asignación.
- Determinar si el objeto tiene un identificador asociado.
- Realizar una operación aritmética.
- Comparar dos valores (numéricos)
- Acceder a un elemento de una lista usando su índice.
- Llamadas a función (el tiempo de ejecución de una función será considerado aparte)
- Return de una función.

Medir las operaciones en función del "tamaño de la entrada"

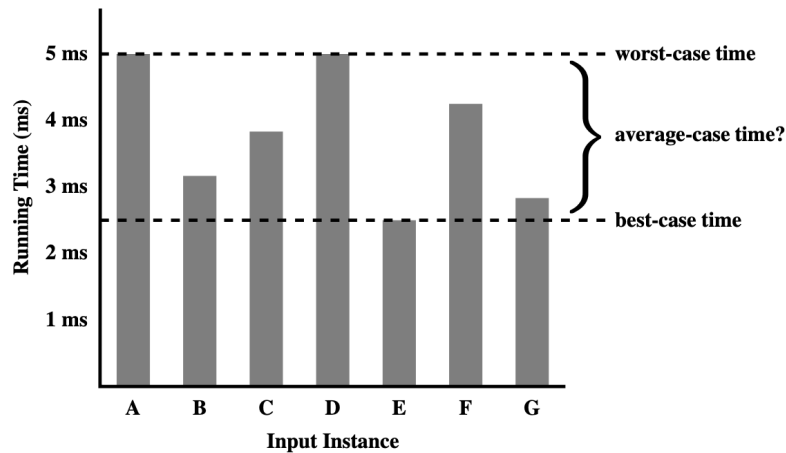
Para capturar el "crecimiento" del tiempo de ejecución de un algoritmo, asociaremos una función $f(n)$ (generalmente llamada $T(n)$) que caracteriza el número de operaciones primitivas realizadas en función de una entrada de tamaño n .

Poner el foco en "el peor caso"

Un algoritmo puede ser sumamente eficiente para un tipo de valores de entrada, y comportarse de manera distinta en otros casos (e.g. Búsqueda vs. Búsqueda Binaria, Alg. de Ordenamiento). Lamentablemente el análisis del "caso promedio" no es una tarea sencilla, generalmente, primero se requiere de un estudio estadístico de los datos de

entrada.

Ejemplo:



El análisis del "peor caso" es mas simple. Permite la generación de mejores algoritmos (aunque menos optimizados, eso es "otro tema").

Algunas funciones matemáticas útiles

La función constante:

$$f(n) = c$$

Cualquier función constante puede ser escrita como:

si $g(n) = 1$, entonces $f(n) = c g(n)$.

La función logarítmico:

$$f(n) = \log_b n$$

La base mas usada es $b = 2$ ya que muchos algoritmos reducen su ejecución reduciendo el problema en dos partes.

La función lineal:

$$f(n) = n$$

Por ejemplo, recorrer un arreglo con un ciclo `for`.

La función N-Log-N:

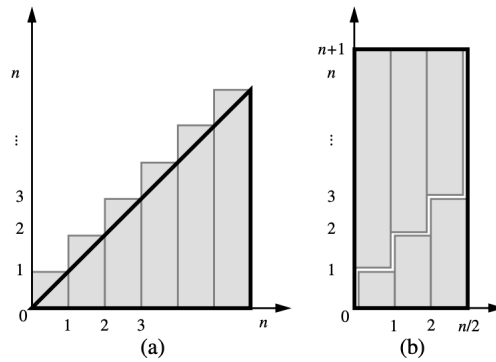
$$f(n) = n \log n$$

La función cuadrática:

$$f(n) = n^2$$

La función cuadrática generalmente aparece cuando tenemos ciclos `for` anidados. En la primer iteración realizamos una operación, luego dos, ..., y así sucesivamente.

$$1 + 2 + 3 + \dots + (n - 2) + (n - 1) + n \approx \frac{n(n + 1)}{2}$$



La función cubica y otros polinomios:

Cubica:

$$f(n) = n^3$$

Polinomios:

$$f(n) = a_0 + a_1 n + a_2 n^2 + a_3 n^3 + \dots + a_d n^d$$

Generalmente el termino de mayor grado es considerado.

Sumas y Series:

$$\sum_{i=a}^b f(i) = f(a) + f(a+1) + f(a+2) + \dots + f(b)$$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

También podemos escribir los polinomios de esta manera:

$$f(n) = \sum_{i=0}^d a_i n^i$$

La función exponencial:

$$f(n) = b^n$$

Sumas Geometricas:

$$\sum_{i=0}^n a^i = 1 + a + a^2 + \dots + a^n = \frac{a^{n+1} - 1}{a - 1}$$

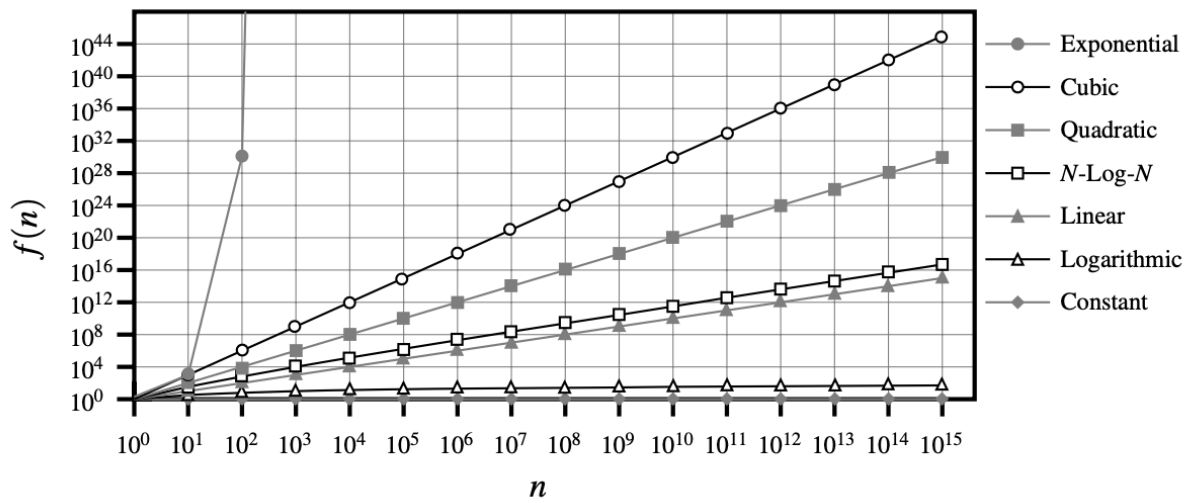
Ejemplo:

$$1 + 2 + 4 + 8 + \dots + 2^{n-1} = 2^n - 1$$

Comparación de estas funciones

constant	logarithm	linear	$n \log n$	quadratic	cubic	exponential
1	$\log n$	n	$n \log n$	n^2	n^3	a^n

Table 3.1: Classes of functions. Here we assume that $a > 1$ is a constant.



Análisis Asintótico

En general estamos interesados en el **crecimiento proporcional** del tiempo que consume un algoritmo dada una entrada de tamaño n .

La notación "Big-Oh"

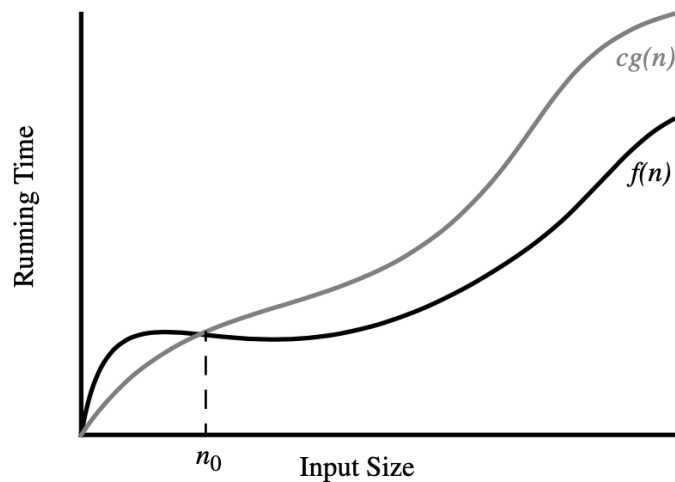


Figure 3.5: Illustrating the “big-Oh” notation. The function $f(n)$ is $O(g(n))$, since $f(n) \leq c \cdot g(n)$ when $n \geq n_0$.

Ejemplo:

La función $8n + 5$ es del orden de ejecución $O(n)$.

Ejemplos:


```
In [ ]: 1 # Tiempo cuadrartico
2 def unique1(S):
3     """Return True if there are no duplicate elements in sequence S."""
4     for j in range(len(S)):
5         for k in range(j+1, len(S)):
6             if S[j] == S[k]:
7                 return False
8     return True
```

```
In [ ]: 1 # Tiempo  $n * \log(n)$ 
2 def unique2(S):
3     """Return True if there are no duplicate elements in sequence S."""
4     temp = sorted(S)
5     for j in range(1, len(temp)):
6         if S[j-1] == S[j]:
7             return False
8     return True
```